

ARE 212 SECTION 4:

Partitioned Regression and Goodness of Fit

Fiona Burlig

February 11, 2016

Tying up loose ends

If you're interested in using `knitr` for your problem set, there's a feature that I haven't introduced you to yet, but which will be incredibly helpful. I'll illustrate it briefly here by using it to make a pretty table. I told you a few weeks ago that `knitr` is partly useful because it's a form of dynamic document. You can use it to display chunks of R code, whose outputs change as you change the input data you're using, but sometimes you don't want to display results in code format. You can instead show off results in line, or in tables - results that also automatically update. Seems magical, right? Let's construct a very brief example, and use our `olsMat` function from last week to illustrate. We'll start by generating some random data:

```
e <- rnorm(10000)
myX <- rnorm(10000)

myY <- 5 + 10*myX + e

# add an intercept to the x matrix
myX <- cbind(1,myX)
```

And loading our function:

```
olsMat <- function(y, X) {
  betahat <- solve(t(X) %*% X) %*% t(X) %*% y
  return(betahat)
}
```

Let's say that I want to report the intercept and the slope of this regression equation. I could do this as follows:

```
results <- olsMat(myY, myX)
results

##           [,1]
##      5.015839
## myX 10.007273
```

That works, but is a little ugly in the PDF. Instead, I can turn this into a nice table, using the `\Sexpr{}` command. I'll write the following $\text{\LaTeX}$ code to turn this into a nice table:

```
% the [h!] tells LaTeX to put this table right here
\begin{table}[h!]
% centering centers things
\centering
% caption: self-explanatory
\caption{OLS Coefficients}
% vspace -- vertical space between caption & table
\vspace{0.5cm}
% begin the table part, ll means 2 left-aligned columns
\begin{tabular}{ll}
% horizontal line
\hline
% ampersands are column breaks, double backslash are line breaks
Variable &  $\hat{\beta}$  \\
\hline
% Sexpr says Evaluate the R stuff. prettyNum lets us specify nr of digits
Intercept & \Sexpr{prettyNum(results[1], digits = 3)} \\
X & \Sexpr{prettyNum(results[2], digits = 3)} \\
\hline
% dont forget to end your tabular and table environments
\end{tabular}
\end{table}
```

Or if you prefer that without the comments:

```
\begin{table}[h!]
\centering
\caption{OLS Coefficients}
\vspace{0.5cm}
\begin{tabular}{ll}
\hline
Variable &  $\hat{\beta}$  \\
\hline
Intercept & \Sexpr{prettyNum(results[1], digits = 3)} \\
X & \Sexpr{prettyNum(results[2], digits = 3)} \\
\hline
\end{tabular}
\end{table}
```

That yields:

Table 1: OLS Coefficients

Variable	$\hat{\beta}$
Intercept	5.02
X	10

Gorgeous. This should help with your problemsetting.

To wrap things up from last time:

Seems like we were able to resolve pretty much everybody's questions during section, but one that came up a couple of times was:

- Q: *What's up with the difference between `select()` and `select_()`? When should I use each one?*

A: The basic idea is this. `select()` and `select_()` take slightly different inputs. For example, we could write `select(autos, mpg)`. To do this with the underscore version, write: `select_(autos, "mpg")`. The reasons behind this are more complicated programming than we need or want for this course.¹ If you're just manipulating data outside of a function, I prefer to use `select()`, because I'm lazy and it saves me typing time. If you're manipulating data *inside* a function, however, you'll need to use `select_()`. This is basically because R understands strings as inputs, but doesn't understand words as inputs to functions. Let's load `dplyr` and give this a try:

```
library(dplyr)
```

If you write a function that looks like:

```
myFxn <- function(data, xvar) {  
  select(data, xvar)  
}
```

and then try to feed it an object like this:

```
#fake random data  
data1 <- rnorm(10000) %>%  
  as.data.frame() %>%  
  tbl_df()  
#column of twos  
data <- mutate(data1, twos = 2)  
  
myFxn(data, twos)  
  
## Error in eval(expr, envir, enclos): object 'twos' not found
```

R will throw you an error. This is because it's looking for the object 'twos', which doesn't exist outside of your dataframe. If instead you modify your function slightly:

```
myBetterFxn <- function(data, xvar) {  
  select_(data, xvar)  
}
```

And now run:

¹If you're curious, Google "R lazyeval".

```
myBetterFxn(data, "twos")

## Source: local data frame [10,000 x 1]
##
##      twos
##      (dbl)
## 1      2
## 2      2
## 3      2
## 4      2
## 5      2
## 6      2
## 7      2
## 8      2
## 9      2
## 10     2
## ..    ...
```

R is happy. Notice that outside of the function environment, these two lines do exactly the same thing:

```
select(data, twos)

## Source: local data frame [10,000 x 1]
##
##      twos
##      (dbl)
## 1      2
## 2      2
## 3      2
## 4      2
## 5      2
## 6      2
## 7      2
## 8      2
## 9      2
## 10     2
## ..    ...
```

```
select_(data, "twos")

## Source: local data frame [10,000 x 1]
##
##      twos
##      (dbl)
## 1      2
## 2      2
```

```
## 3      2
## 4      2
## 5      2
## 6      2
## 7      2
## 8      2
## 9      2
## 10     2
## ..    ...
```

Hopefully that helps.

Okay, on we go.

Oh, there goes gravity!

This section, we'll apply our newfound R skills to thinking about the (incredibly powerful) Frisch-Waugh-Lovell Partitioned Regression theorem. We'll also play around with R^2 a little bit - though we might not have time to get all the way through this material. You'll have it as a guide on your problem set, though.

As usual, we'll start by loading our favorite packages.

```
library(haven)
library(lfe)
```

Let's install and grab two more packages - one we'll use now, and one we'll use later:

```
install.packages('readr')
install.packages('MASS')
```

```
library(readr)
library(MASS)

## Warning: package 'MASS' was built under R version 3.2.2
```

Let's also set up our working directory, and load our favorite dataset²:

```
#setwd("/users/fwilkes1/Desktop/Dropbox/UC Berkeley/2015-2016/ARE 212/Section 1")
autos <- read_dta("autos.dta")
```

We can also load data from a CSV (as you'll do on your problem set) directly into a `tbl_df` object like this (this uses the `readr` package):

²Setting your working directory is unnecessary if you're running `knitr` and saving your `.Rnw` file in the same folder as your data. I've commented this line out in my own code.

```
autos <- read_csv("autos.csv")
```

We'll also bring in (a slightly modified version of) our OLS function from last week:

```
noIntOLS <- function(data, y, X) {  
  # grab y, put it into the form we want  
  ydata <- data %>%  
    select_(y) %>%  
    as.matrix()  
  # grab x, put it into the form we want  
  xdata <- data %>%  
    select_(.dots = X) %>%  
    as.matrix()  
  
  # solve for beta hat  
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata  
  # and name it nicely:  
  names(betahat) <- X  
  return(betahat)  
}
```

That should look familiar to you, but with one change - this version of OLS doesn't automatically add an intercept (dun dun dunnnn). For now, this function is pretty nice if we're going to be just running OLS - but we're often going to want to run multiple functions put together, starting from a `tbl_df` object. So let's write a function that will grab variables from `tbl_df` objects and turn them into the matrix objects that we'll be mostly working with in this class:

```
tbl_dfGrabber <- function(data, varnames) {  
  matrixObject <- data %>%  
    select_(.dots = varnames) %>%  
    as.matrix()  
}
```

Now we can streamline our OLS function nicely:

```
noIntOLS <- function(data, y, X) {  
  ydata <- tbl_dfGrabber(data, y)  
  xdata <- tbl_dfGrabber(data, X)  
  # solve for beta hat  
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata  
  names(betahat) <- X  
  return(betahat)  
}
```

Beautiful. That's much shorter code, making it more legible - and now we have `ydata` and `xdata` objects that we can play around with.

Since we will be adding an intercept to some of our models, let's go ahead and add a column of ones to our `autos` dataframe:

```

autos <- mutate(autos, ones = 1)

head(autos$ones)

## [1] 1 1 1 1 1 1

```

Frisch-Whaaaaa?

The Frisch-Waugh-Lovell partitioned regression theorem is one of the most powerful tools in the applied econometrician's bag of tricks, and in my (not-so?) humble opinion, it's often under-appreciated. What does this result say? Suppose we have the following model:

$$\mathbf{y} = \mathbf{X}_1\boldsymbol{\beta}_1 + \mathbf{X}_2\boldsymbol{\beta}_2 + \boldsymbol{\varepsilon}$$

The theorem says that we can estimate $\mathbf{b}_1$ as:

$$\mathbf{b}_1 = (\mathbf{X}_1'\mathbf{X}_1)^{-1}\mathbf{X}_1'\mathbf{y} - (\mathbf{X}_1'\mathbf{X}_1)^{-1}\mathbf{X}_1'\mathbf{X}_2\mathbf{b}_2$$

Define $\mathbf{M}_2$ as the residual-maker matrix, such that post-multiplying it by another object will recover the residuals from regressing that object onto $\mathbf{X}_2$. Then we can further define $\mathbf{X}_1^* = \mathbf{M}_2\mathbf{X}_1$, and $\mathbf{y}^* = \mathbf{M}_2\mathbf{y}$. We can now write:

$$\mathbf{b}_1 = (\mathbf{X}_1^{*'}\mathbf{X}_1^*)^{-1}\mathbf{X}_1^{*'}\mathbf{y}^*$$

That is, we can recover $\mathbf{b}_1$ either by running a regression of $\mathbf{y}$ on $\mathbf{X}_1$ and subtracting off a “correction factor”, or by running a regression of a *residualized* $\mathbf{y}$ on a residualized $\mathbf{X}_1$. We can obviously do a similar procedure to come up with $\mathbf{b}_2$.

For a full derivation of this, see Max's course notes.

How the sausage is made: regression and residuals

The first insight that we should glean from Frisch-Waugh-Lovell is mechanical. We will recover the same $\hat{\boldsymbol{\beta}}_2$ coefficient from this regression:

$$\mathbf{y} = \mathbf{X}_1\boldsymbol{\beta}_1 + \mathbf{X}_2\boldsymbol{\beta}_2 + \boldsymbol{\varepsilon}$$

As from the following procedure:

1. Regress $\mathbf{y}$ on $\mathbf{X}_1$.
2. Save the residuals - call them $\mathbf{e}_{\mathbf{y}\mathbf{X}_1}$.
3. Regress each column of $\mathbf{X}_2$ on $\mathbf{X}_1$.
4. Save the residuals - call them $\mathbf{e}_{\mathbf{X}_1\mathbf{X}_2}$.
5. Regress $\mathbf{e}_{\mathbf{y}\mathbf{X}_1}$ on $\mathbf{e}_{\mathbf{X}_1\mathbf{X}_2}$.

Mind. Blown. Let's convince ourselves that this is true. First, let's run our favorite regression: car price on an intercept, MPG, and weight. To be clear, let's call price $\mathbf{y}$, the intercept $\mathbf{X}_1$, and MPG and weight $\mathbf{X}_2$.

```

betahatOLS <- noIntOLS(autos, "price", c("ones", "mpg", "weight"))
betahatOLS

##           price
## ones    1946.068668
## mpg      -49.512221
## weight     1.746559
## attr(,"names")
## [1] "ones"  "mpg"   "weight"

```

Now let's write a function to store residuals (this should look similar to the first part of our R^2 function above:

```

residuals <- function(data, y, X) {
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)

  betaHat <- noIntOLS(data, y, X)
  yHat <- xdata %*% betaHat
  resids <- ydata - yHat
  return(resids)
}

```

Okay. Our five-step program begins. First regress price on the intercept and save the residuals.

```
residualsPriceOnes <- residuals(autos, "price", "ones")
```

Next regress MPG and weight on the intercept, and save the residuals.

```

residualsMPGInt <- residuals(autos, "mpg", "ones")
residualsWeightInt <- residuals(autos, "weight", "ones")

# combine all three residuals into a tbl_df
residualData <- cbind(residualsPriceOnes, residualsMPGInt, residualsWeightInt) %>%
  as.data.frame() %>%
  tbl_df

names(residualData) <- c("rprice1", "rMpg1", "rWeight1")

```

Finally, regress the residuals from the first regression on the residuals from the second two:

```

betahatFWL <- noIntOLS(residualData, "rprice1", c("rMpg1", "rWeight1"))
# Compare results:
betahatOLS[-1]

##      mpg      weight
## -49.512221  1.746559

betahatFWL

```

```
##           rprice1
## rMpg1      -49.512221
## rWeight1    1.746559
## attr(,"names")
## [1] "rMpg1"      "rWeight1"
```

Mind. Officially. Blown. This is awesome! We can use this for all kinds of things. One example: when you run a regression with many fixed effects or other variables, you're asking your computer to invert a very large matrix. Applying Frisch-Waugh-Lovell allows us to break that big matrix into partitions, and still get the right $\hat{\beta}$. This is what `lfe()`, and its counterpart in Stata, `-reghdfe-`, are doing when you throw a bunch of fixed effects on the right-hand side.³ If that didn't make sense to you, or you don't know what a fixed effect is, don't worry - just know that one benefit of Frisch-Waugh-Lovell is easing computational needs.

It's cool that we can recover the correct point estimates this way - but we can actually use Frisch-Waugh-Lovell to highlight two other key insights.⁴

Forgetting something? Omitted variable bias

Remember this line from the math above?:

$$\mathbf{b}_1 = (\mathbf{X}'_1 \mathbf{X}_1)^{-1} \mathbf{X}'_1 \mathbf{y} - (\mathbf{X}'_1 \mathbf{X}_1)^{-1} \mathbf{X}'_1 \mathbf{X}_2 \mathbf{b}_2$$

This says that we can recover $\mathbf{b}_1$ from a regression of $\mathbf{y}$ on $\mathbf{X}_1$, minus a correction factor, $(\mathbf{X}'_1 \mathbf{X}_1)^{-1} \mathbf{X}'_1 \mathbf{X}_2 \mathbf{b}_2$. If $\mathbf{X}_1$ and $\mathbf{X}_2$ are orthogonal, this correction factor is equal to zero, and we can recover $\mathbf{b}_1$ from a regression of $\mathbf{y}$ only on $\mathbf{X}_1$. Of course, under orthogonality, we can similarly recover $\mathbf{b}_2$ just by regressing $\mathbf{y}$ on $\mathbf{X}_2$.⁵

Let's confirm this in R. First, we'll generate some fake data:

```
x1 <- rnorm(10000)
x2 <- rnorm(10000)
e <- rnorm(10000)
ones <- rep(1, 10000)
# DGP!
y <- 2*x1 - 3*x2 + e

randomData <- cbind(y, ones, x1, x2) %>%
  as.data.frame() %>%
  tbl_df()
```

Let's run a regression of $\mathbf{y}$ on $\mathbf{X}_1$ and $\mathbf{X}_2$ to make sure that we get the right thing:

³Note that, if you build your own code to do this, that your standard errors will not be correct - you'll have to apply a degrees-of-freedom correction on your final regression. But that's a story for another day.

⁴These are definitely not the only two insights from Frisch-Waugh-Lovell - it'll serve you well over your career to spend some time really engaging with this theorem.

⁵In partition-land, if $\mathbf{X}_1$ and $\mathbf{X}_2$ are uncorrelated, the residual from the regression of $\mathbf{X}_2$ on $\mathbf{X}_1$ will be $\mathbf{X}_2$ itself (since the residual is all of the variation in $\mathbf{X}_2$ that is unexplained by $\mathbf{X}_1$). This means that steps 3-5 above would be equivalent to running a regression of $\mathbf{e}_{y\mathbf{X}_1}$ on $\mathbf{X}_2$, which will recover $\hat{\beta}_2$. Don't believe me? Good - do the linear algebra yourself.

```
(fullTest <- noIntOLS(randomData, "y", c("ones", "x1", "x2")))

##              y
## ones -0.01827315
## x1    1.99377940
## x2   -3.00382798
## attr(,"names")
## [1] "ones" "x1"  "x2"
```

Pretty darn close. (We'd be right on the money if we had more observations). So now let's see if we can recover each coefficient individually:

```
(x1Test <- noIntOLS(randomData, "y", c("ones", "x1")))

##              y
## ones 0.01092387
## x1   1.97047563
## attr(,"names")
## [1] "ones" "x1"
```

```
(x2Test <- noIntOLS(randomData, "y", c("ones", "x2")))

##              y
## ones -0.005667285
## x2   -2.988035371
## attr(,"names")
## [1] "ones" "x2"
```

Excellent. But what if $\mathbf{X}_1$ and $\mathbf{X}_2$ *aren't* orthogonal?

```
# create a variance-covariance matrix with 1s on the diag and
# 0.95 on the off-diag

sigmaMat <- matrix(data = c(1,0.95,0.95,1), nrow = 2)
# create x1 and x2 JOINTLY, drawing from a multivariate normal
# mvrnorm() with means 5, 10, and sigma (the cov structure) as above
xmat <- mvrnorm(n = 10000, mu = c(5,10), Sigma = sigmaMat, empirical = TRUE) %>%
  matrix(nrow = 10000)
e <- rnorm(10000)
ones <- rep(1, 10000)
# DGP!
y <- 1 - 3*xmat[,1] + 2*xmat[,2] + e

randomDataNonOrth <- cbind(y, ones, xmat[,1], xmat[,2]) %>%
  as.data.frame() %>%
  tbl_df()
names(randomDataNonOrth) <- c("y", "ones", "x1", "x2")
```

Test again:

```
(fullTestNonOrth <- noIntOLS(randomDataNonOrth, "y", c("ones", "x1", "x2")))  
  
##           y  
## ones  0.7178564  
## x1   -3.0433190  
## x2    2.0493795  
## attr(,"names")  
## [1] "ones" "x1"  "x2"
```

Still getting the right coefficients. Good! What about individually?

```
(x1TestNonOrth <- noIntOLS(randomDataNonOrth, "y", c("ones", "x1")))  
  
##           y  
## ones 11.477099  
## x1   -1.096408  
## attr(,"names")  
## [1] "ones" "x1"
```

```
(x1TestNonOrth <- noIntOLS(randomDataNonOrth, "y", c("ones", "x2")))  
  
##           y  
## ones 14.4127917  
## x2   -0.8417735  
## attr(,"names")  
## [1] "ones" "x2"
```

Uh oh. Looks like we're no longer recovering the true values.⁶ These regressions are suffering from *omitted variable bias*. In effect, that “correction factor” from the above formula turns out to be super important. If there's a part of the data generating process that you don't model, but it's correlated with a part that you *do* model, you're going to get the wrong answer. This should make you deeply uncomfortable, and very afraid.⁷

When more isn't always better: bad controls

We can also show another frightening fact, and unpack it using Frisch-Waugh-Lovell logic. To do this, we'll steal an example from Sol Hsiang.⁸ Let's start by generating the relevant data according to the following DGP:

$$y = 5 + 2X$$

⁶I thought I was being *so* clever when I came up with this example. Turns out it's verbatim from my own ARE 212 midterm. Burned into some hidden corner of my brain - and soon into yours as well - though I will take credit for couching it in FWL logic.

⁷This is not a causal inference class - ARE 213 covers that - but as real-world applied econometricians, we care a *lot* about recovering coefficients without bias. Omitted variables are a big problem.

⁸The accompanying blog post is [here](#). In general, I highly recommend the Fight Entropy blog - Sol and Jesse now have real jobs and have basically stopped posting, but the archives contain a bunch of really interesting stuff.

```
set.seed(10)

# True DGP
X <- rnorm(10000)
Y <- 5 + 2*X
```

Now let's add a third variable into the mix, $Z = X$. Note that Z is not actively involved in the DGP:

```
Z <- X
```

To make this example work out in practice, we need to make sure that things aren't perfectly collinear. To do that, we'll add a little bit of random noise to each variable (you can think of this as the difference between "truth" and what the econometrician observes):

```
# Observation error (small, and normally distributed)
# we need this to make sure things aren't perfectly collinear
e.X <- 0.1*rnorm(10000)
e.Z <- 0.1*rnorm(10000)
e.Y <- 0.1*rnorm(10000)

# What the econometrician actually observes
x <- X + e.X
z <- Z + e.Z
y <- Y + e.Y

# our observed data in tbl_df form
solData <- cbind(y, 1, x, z) %>%
  as.data.frame() %>%
  tbl_df

names(solData) <- c("y", "ones", "x", "z")
```

Now let's actually run the regression of y on x :

```
(solXY <- noIntOLS(solData, "y", c("ones", "x")))
```

```
##           y
## ones 5.002418
## x    1.981040
## attr(,"names")
## [1] "ones" "x"
```

Boom - got it. That's the true relationship. But now what happens if we introduce z into our regression?

```
(solXYZ <- noIntOLS(solData, "y", c("ones", "x", "z")))
```

```
##           y
## ones 4.9998862
```

```
## x      1.0009267
## z      0.9901564
## attr(,"names")
## [1] "ones" "x"      "z"
```

Wait, what? z wasn't part of our DGP at all - so this isn't the omitted variables problem we had before. This is instead a version of the “bad controls” problem: even though z wasn't part of our DGP, it *is* highly correlated with x . When we include z in our regression of y on x , we no longer recover the true relationship between y and x , because of Frisch-Waugh-Lovell. Running this regression, we might incorrectly conclude that the true effect of x on y is only 1, rather than 2 - this is wrong. Since x and z aren't orthogonal, we're “misattributing” x 's explanatory power to z .

Punchline: Frisch-Waugh-Lovell is something you should keep in mind when deciding what variables to include (or not) in a regression.

$[R^2]$ says that the chances of survival are 725 to 1

Actually, Artoo has been known to make mistakes from time to time...oh dear.⁹ R^2 is one measure of goodness of fit of a regression line - but as Max will tell you, it's often abused. In this section, we'll first show you what the various measures of R^2 are actually calculating. We'll then write our own R function to calculate each kind, and use that to figure out what R's canned routines are doing under the hood. Finally, we'll demonstrate why overreliance on R^2 isn't always a good thing.

Intuition: Hey! What's the big idea?

If you read that out loud in your head [like this](#), you are also a child of the 90s. Congratulations. We'll start off by thinking about what R^2 is trying to do. There are three types of R^2 that we'll deal with today.

- The *uncentered* R^2 , R_{uc}^2 , is a measure of how much of the variance in the data our model (read: our OLS regression line) explains relative to a hyperplane (read: line) where $y = 0$.
- The *centered* R^2 , just plain R^2 , is a measure of how much of the variance in the data our model explains relative to a hyperplane where $y = \bar{y}$.
- The *adjusted* R^2 , $\bar{R}^2$ takes the centered R^2 and applies a penalty for adding covariates.

R^2 , formally

We can also write all of these down in math, starting with the uncentered version:

$$R_{UC}^2 \equiv \frac{\hat{\mathbf{y}}'\hat{\mathbf{y}}}{\mathbf{y}'\mathbf{y}} = 1 - \frac{\mathbf{e}'\mathbf{e}}{\mathbf{y}'\mathbf{y}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - 0)^2} = 1 - \frac{\text{SSR}}{\text{SST}_0}$$

This is the (very general) formula for the uncentered R_{uc}^2 . Notice that the first equality comes from the fact that $\mathbf{y}'\mathbf{y} = \hat{\mathbf{y}}'\hat{\mathbf{y}} + \mathbf{e}'\mathbf{e}$, and the second from converting to summation notation. The final equality just represents a nice way of thinking about what the uncentered R_{uc}^2 is actually measuring: SSR is the sum of squared residuals, and

⁹Don't get this joke? [Here's some money](#). [Go see a Star War](#).

SST_0 is the total sum of squares *relative to zero* - note that this is different from what's in Max's notes.

The formula for the centered R^2 looks very similar:

$$R^2 \equiv 1 - \frac{\mathbf{e}'\mathbf{e}}{\mathbf{y} *' \mathbf{y} *} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{SSR}{SST}$$

Notice that now we *are* using the SST from Max's notes. Also, remember that $\mathbf{y} * -\mathbf{A}\mathbf{y}$, the demeaned version of $\mathbf{y}$. This expression for R^2 is a little different than the one in the notes - it allows for a model without an intercept.¹⁰

Finally, the adjusted $\bar{R}^2$ is just a transformation of the centered R^2 that penalizes the addition of additional covariates:

$$\bar{R}^2 \equiv 1 - \frac{n-1}{n-k}(1 - R^2)$$

where n is the number of observations and k is the number of covariates. Note that, thankfully, if $k = 1$, this collapses down to the centered R^2 .

I'm tired of math. Let's write some code!

MTV's The Real World: R^2

We've already written most of what we'll need to calculate R^2 above. Let's make one more function to generate an n -by- n demeaning matrix, $\mathbf{A}$.¹¹

```
demeaner <- function(n) {  
  # implements  $A = I_n - (1/n)ii'$  from the notes  
  demeanMat <- diag(n) - (1/n)  
  return(demeanMat)  
}
```

Note that we could've also expressed that penultimate line as:

```
demeanMat <- diag(n) - (1/n) * ones %*% t(ones)
```

But we don't actually have to. Why? Because R is smart enough to realize that since `diag(n)` is an n -by- n object, it subtracts $\frac{1}{n}$ elementwise.¹²

Now we can actually write our R^2 function! Whoo!

```
rSquared <- function(data, y, X) {  
  # set up some objects we care about  
  n <- nrow(data)  
  k <- length(X)  
  ydata <- tbl_dfGrabber(data, y)
```

¹⁰Greene 3.5.2 has a nice discussion of why the version in the lecture notes won't work without a constant term.

¹¹Raise your hand if you've ever been personally demeaned by the $\mathbf{A}$ matrix.

¹²Unclear? Play around with it yourself. Basically what's happening is that `ones %*% t(ones)` creates an n -by- n matrix of ones - premultiplying it by $(1/n)$ yields an n -by- n matrix of $(1/n)$ s. But when you subtract a scalar from a vector or matrix in R, you're already doing element-wise subtraction - so this additional step is unnecessary.

```

xdata <- tbl_dfGrabber(data, X)

# run OLS

# yes, this is a little redundant
# we could've written a simpler OLS fcn
# that just starts with matrix objects
# but shh, I like this one
betaHat <- noIntOLS(data, y, X)

# use the output of OLS to create things we want
yHat <- xdata %*% betaHat
e <- ydata - yHat

# RSquared objects
### UNCENTERED
SSR <- t(e) %*% e
SST.0 <- t(ydata) %*% ydata # ==sum((y - 0)^2)
R2.unc <- 1 - SSR / SST.0

### CENTERED
A <- demeaner(n)
yStar <- A %*% ydata
SST <- t(yStar) %*% yStar #== sum((y - ybar)^2)
R2.cen <- 1 - SSR / SST

### ADJUSTED
R2.adj <- 1 - ((n-1)/(n-k)) * (1-R2.cen)

# Put together the output
rsqOutput <- list(betaHat, R2.unc, R2.cen, R2.adj)
names(rsqOutput) <- c("betaHat", "R2.unc", "R2.cen", "R2.adj")
return(rsqOutput)
}

```

We now have a pretty R^2 function. Let's take a look at the output by testing this bad boy on our favorite regression: price on an intercept, and MPG:

```

#Note that we now need to manually tell OLS to include an intercept!
myRsqr <- rSquared(autos, "price", c("ones", "mpg"))

myRsqr

## $betaHat
##           price
## ones 11253.0607
## mpg  -238.8943
## attr(,"names")

```

```
## [1] "ones" "mpg"
##
## $R2.unc
##      price
## price 0.856253
##
## $R2.cen
##      price
## price 0.2195829
##
## $R2.adj
##      price
## price 0.2087437
```

Whoa! What's this crazy object?!?! Notice that it contains a 3 vector (our OLS output), as well as three 1 vectors (R_{uc}^2 , R^2 , and $\bar{R}^2$). Crazy. This is called a *list* - a data object that contains within it different types of data objects.¹³ We can also grab different elements of this list, either by number or by name:

```
myRsqr[1]

## $betaHat
##      price
## ones 11253.0607
## mpg   -238.8943
## attr(,"names")
## [1] "ones" "mpg"

myRsqr$betaHat

##      price
## ones 11253.0607
## mpg   -238.8943
## attr(,"names")
## [1] "ones" "mpg"
```

We ain't messing around here in ARE 212. But before we get too carried away, let's make sure that our R^2 function is actually doing the right thing by checking it against `feelm()`:

```
cannedRsqr <- (autos %>%
  feelm(formula = price ~ mpg) %>%
  summary())$r2
```

By comparing it to our own function, looks like this is returning the *centered* R^2 . Good stuff. We can actually also ask `feelm()` to show us the adjusted R^2 :

¹³Yes, wiseass, you can make a list of lists.

```
cannedRsqrAdj <- (autos %>%
  felm(formula = price ~ mpg) %>%
  summary())$r2adj
```

Awesome. But it won't return the uncentered R_{uc}^2 . Rude. Oh well. Let's try running our `rSquared()` function again, this time *without* an intercept on the right-hand side.¹⁴

```
myScaryRsqr <- rSquared(autos, "price", c("mpg"))

cannedScaryRsqr <- (autos %>%
  #notice the 0+ tells felm to use no intercept
  felm(formula = price ~ 0 + mpg) %>%
  summary())$r2

cannedScaryRsqrAdj <- (autos %>%
  felm(formula = price ~ 0 + mpg) %>%
  summary())$r2adj
```

```
myScaryRsqr

## $betaHat
##      price
## mpg 253.6302
## attr(,"names")
## [1] "mpg"
##
## $R2.unc
##      price
## price 0.6718225
##
## $R2.cen
##      price
## price -0.7817091
##
## $R2.adj
##      price
## price -0.7817091
```

```
cannedScaryRsqr

## [1] -0.7817091

cannedScaryRsqrAdj

## [1] -0.8061161
```

¹⁴Be afraid. Be very afraid.

What the eff? Negative R^2 ? Actually, that makes sense. R^2 compares our model, $\mathbf{y} = \mathbf{X}\beta + \varepsilon$ with no intercept to $\mathbf{y} = \bar{\mathbf{y}}$. Since $\bar{\mathbf{y}}$ is a much better predictor of $\mathbf{y}$ than $\mathbf{X}\beta$, $\text{SSR} > \text{SST}$, so $R^2 < 0$. Notice that we get the same thing with the canned routine - but! If we'd used the older, less fancy canned routine, `lm()`, we'd have seen something different:

```
cannedScaryRsqrOther <- (autos %>%
  lm(formula = price ~ 0 + mpg) %>%
  summary())$r.squared

cannedScaryRsqrOther

## [1] 0.6718225
```

This time, we get a “centered” R^2 value of 0.67. That looks awfully close to our *uncentered* estimate - what gives? Because `lm()` sees a model with no intercept, it automatically calculates R_{uc}^2 rather than R^2 - a bait and switch! Booooo. This is why it's good to know what canned routines are actually doing. But TL;DR: include an intercept in your model, and all will be well in the world.

It's a trap! Overfitting and R^2

Remember how we keep telling you that maximizing R^2 isn't a good way of judging your model? Here's a good part of the reason why. Let's add some variables to our regression. But instead of real data, let's add a bunch of random noise. First, we'll write a function to create a matrix of noise.

```
#don't forget - randomness --> set seed!
set.seed(1)

noisyColumns <- function(n, k) {
  # generate random noise vector
  noisyMat <- runif(n*k) %>%
    # transform to matrix
    matrix(nrow = n, ncol = k)
  # use sapply to name the columns col1 ... colk
  colnames(noisyMat) <- sapply(1:k, function(x) {
    paste("col", x, sep = "")
  })
  #output the matrix
  return(noisyMat)
}
```

Now let's create 50 noisy columns and add them to our `autos` data:

```
rows <- nrow(autos)
nrVars <- 50

# create a bunch of random noise and turn it into a tbl_df
randomCols <- noisyColumns(rows, nrVars) %>%
  as.data.frame() %>%
```

```

      tbl_df()
#merge it onto our autos dataset
bigAutos <- bind_cols(autos, randomCols)
#check it:
bigAutos

## Source: local data frame [74 x 63]
##
##           make price   mpg rep78 headroom trunk weight length
##           (chr) (int) (int) (int)   (dbl) (int)  (int)  (int)
## 1    AMC Concord  4099   22     3     2.5   11   2930   186
## 2    AMC Pacer   4749   17     3     3.0   11   3350   173
## 3    AMC Spirit  3799   22    NA     3.0   12   2640   168
## 4 Buick Century  4816   20     3     4.5   16   3250   196
## 5 Buick Electra  7827   15     4     4.0   20   4080   222
## 6 Buick LeSabre 5788   18     3     4.0   21   3670   218
## 7   Buick Opel   4453   26    NA     3.0   10   2230   170
## 8   Buick Regal  5189   20     3     2.0   16   3280   200
## 9 Buick Riviera 10372   16     3     3.5   17   3880   207
## 10 Buick Skylark 4082   19     3     3.5   13   3400   200
## ..           ...     ...     ...     ...     ...     ...     ...
## Variables not shown: turn (int), displacement (int), gear_ratio
## (dbl), foreign (int), ones (dbl), col1 (dbl), col2 (dbl), col3
## (dbl), col4 (dbl), col5 (dbl), col6 (dbl), col7 (dbl), col8
## (dbl), col9 (dbl), col10 (dbl), col11 (dbl), col12 (dbl),
## col13 (dbl), col14 (dbl), col15 (dbl), col16 (dbl), col17
## (dbl), col18 (dbl), col19 (dbl), col20 (dbl), col21 (dbl),
## col22 (dbl), col23 (dbl), col24 (dbl), col25 (dbl), col26
## (dbl), col27 (dbl), col28 (dbl), col29 (dbl), col30 (dbl),
## col31 (dbl), col32 (dbl), col33 (dbl), col34 (dbl), col35
## (dbl), col36 (dbl), col37 (dbl), col38 (dbl), col39 (dbl),
## col40 (dbl), col41 (dbl), col42 (dbl), col43 (dbl), col44
## (dbl), col45 (dbl), col46 (dbl), col47 (dbl), col48 (dbl),
## col49 (dbl), col50 (dbl)

```

Now let's run a bunch of regressions, progressively adding in one covariate at a time, and see what happens to the R^2 and $\bar{R}^2$:

```

# Create a vector of column names
# ("col1", "col2", ...)
colNames <- sapply(1:50, function(x) {
  name <- paste("col", x, sep = "")
})

# run our r^2 function
rsquareds <- sapply(1:50, function(x) {
  # step 1: create a new X vector, adding 1 col each time

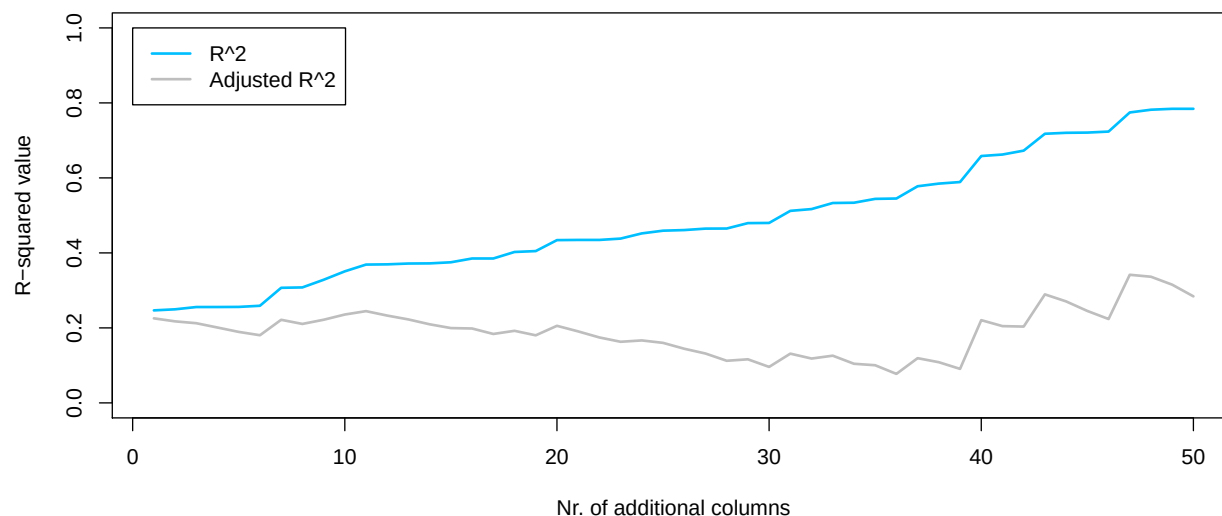
```

```
Xvec <- c("ones", "mpg", colNames[seq(x)])
# step 2: run the r^2 fcn
rSquared(bigAutos, "price", Xvec)
})
```

Make a picture:

```
# Call plot on the data we're interested in (2 variables here: X then Y)

# make a picture!
# plot the regular R^2 and label (ylim sets the y axis limits)
plot(x = 1:50, y = rsquareds[3,], type = "l", lwd = 2, col = "deepskyblue",
xlab = "Nr. of additional columns", ylab = "R-squared value", ylim=c(0,1))
# add the adjusted R^2
lines(x = 1:50, y = rsquareds[4,], type = "l", lwd = 2, col = "gray")
# and a legend, where lwd specifies legend line width and color legend colors
legend(0,1,c("R^2","Adjusted R^2"), lwd = c(2,2), col = c("deepskyblue","gray"))
```



Check it - as we add covariates, our R^2 goes from around 0.2 to around 0.8 - so our model must be better, right? We know that it isn't! Don't be an R^2 maximizer. Put enough random noise into a model and you're going to end up explaining *something*.¹⁵ The good news is that our $\bar{R}^2$ stays about the same, suggesting that our additional columns aren't actually doing much.¹⁶

That's it for this week. We'll pick up next week with hypothesis testing.

¹⁵When you add too many columns (more than we have observations), things start to break. Why? Ponder your rank conditions.

¹⁶What do you think would happen to $\bar{R}^2$ as we add "too many" columns? Ponder your rank conditions again.